

ASSIGNMENT-5.1 & 6

Hall Ticket: 2303A510i2

Name: Kasuri Rohith

Batch:23

Task 1:

Employee Data: Create Python code that defines a class named Employee with the following attributes: empid, empname, designation, basic_salary, and exp. Implement a method display_details() to print all employee details. Implement another method calculate_allowance() to determine additional allowance based on experience:

- If $\text{exp} > 10$ years $\rightarrow$ allowance = 20% of basic_salary
- If $5 \leq \text{exp} \leq 10$ years $\rightarrow$ allowance = 10% of basic_salary
- If $\text{exp} < 5$ years $\rightarrow$ allowance = 5% of basic_salary

Finally, create at least one instance of the Employee class, call the display_details() method, and print the calculated allowance.

Code:

```
1 class Employee:
2     def __init__(self, empid, empname, designation, basic_salary, exp):
3         self.empid = empid
4         self.empname = empname
5         self.designation = designation
6         self.basic_salary = basic_salary
7         self.exp = exp
8
9     def display_details(self):
10        print(f"Employee ID:{self.empid}")
11        print(f"Employee Name:{self.empname}")
12        print(f"Designation:{self.designation}")
13        print(f"Basic Salary:{self.basic_salary}")
14        print(f"Experience:{self.exp} years")
15
16    def calculate_allowance(self):
17        if self.exp > 10:
18            allowance = 0.20 * self.basic_salary
19        elif 5 <= self.exp <= 10:
20            allowance = 0.10 * self.basic_salary
21        else:
22            allowance = 0.05 * self.basic_salary
23
24        print(f"Allowance:{allowance}")
25        print(f"Total Salary:{self.basic_salary+allowance}")
26
27 empobj1 = Employee(101,"Alice","Manager",80000,12)
28 empobj1.display_details()
29 empobj1.calculate_allowance
30 print()
31 empobj2 = Employee(102,"Vicky","HR",160000,20)
32 empobj2.display_details()
33 empobj2.calculate_allowance
```

Output:

```
PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/OneDrive/Desktop/ai/day7.py
Employee ID:101
Employee Name:Alice
Designation:Manager
Basic Salary:80000
Experience:12 years

Employee ID:102
Employee Name:Vicky
Designation:HR
Basic Salary:160000
Experience:20 years
PS C:\Users\rohit\OneDrive\Desktop\ai>
```

Task 2:

Electricity Bill Calculation- Create Python code that defines a class named ElectricityBill with attributes: customer_id, name, and units_consumed. Implement a method display_details() to print customer details, and a method calculate_bill() where:

- Units $\leq 100 \rightarrow ₹5$ per unit
- 101 to 300 units $\rightarrow ₹7$ per unit
- More than 300 units $\rightarrow ₹10$ per unit

Create a bill object, display details, and print the total bill amount.

Code:

```
35 class ElectricityBill():
36     def __init__(self, customer_id, name, units_consumed):
37         self.customer_id = customer_id
38         self.name = name
39         self.units_consumed = units_consumed
40
41     def display_details(self):
42         print("Customer ID:", self.customer_id)
43         print("Customer Name:", self.name)
44         print("Units Consumed:", self.units_consumed)
45
46     def calculate_bill(self):
47         if self.units_consumed <= 100:
48             bill_amount = self.units_consumed * 5
49         elif 101 <= self.units_consumed <= 300:
50             bill_amount = (100 * 5) + (self.units_consumed - 100) * 7
51         else:
52             bill_amount = (100 * 5) + (200 * 7) + (self.units_consumed - 300) * 10
53         return bill_amount
54
55 bill = ElectricityBill(101, "Ramesh", 350)
56 bill.display_details()
57 amount = bill.calculate_bill()
58 print(f"Total Bill Amount: {amount}")
59
```

Output:

```
PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/OneDrive/Desktop/ai/day7.py
Customer ID: 101
Customer Name: Ramesh
Units Consumed: 350
Total Bill Amount: 2400
PS C:\Users\rohit\OneDrive\Desktop\ai>
```

Task 3:

Product Discount Calculation- Create Python code that defines a class named Product with attributes: product_id, product_name, price, and category. Implement a method display_details() to print product details. Implement another method calculate_discount() where:

- Electronics → 10% discount
- Clothing → 15% discount
- Grocery → 5% discount

Create at least one product object, display details, and print the final price after discount.

Code:

```
class Product:
    def __init__(self, product_id, product_name, price, category):
        self.product_id = product_id
        self.product_name = product_name
        self.price = price
        self.category = category

    def display_details(self):
        print(f"Product ID:{self.product_id}")
        print(f"Product Name:{self.product_name}")
        print(f"Price:{self.price}")
        print(f"Category:{self.category}")

    def calculate_discount(self):
        if self.category.lower() == "electronics":
            discount = 0.10 * self.price
        elif self.category.lower() == "clothing":
            discount = 0.15 * self.price
        else:
            discount = 0.05 * self.price

        print(f"Discount Amount:{discount}")
        print(f"Price after Discount:{self.price - discount}")

prod1 = Product(301, "Laptop", 60000, "Electronics")
prod1.display_details()
prod1.calculate_discount()
```

Output:

```

PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/OneDrive/Desktop/ai/day7.py
Product ID:301
Product Name:Laptop
Price:60000
Category:Electronics
Discount Amount:6000.0
Price after Discount:54000.0
PS C:\Users\rohit\OneDrive\Desktop\ai>

```

Task 4:

Book Late Fee Calculation- Create Python code that defines a class named LibraryBook with attributes: book_id, title, author, borrower, and days_late. Implement a method display_details() to print book details, and a method calculate_late_fee() where:

- Days late $\leq 5 \rightarrow ₹5$ per day
- 6 to 10 days late $\rightarrow ₹7$ per day
- More than 10 days late $\rightarrow ₹10$ per day

Create a book object, display details, and print the late fee

Code:

```

class LibraryBook:
    def __init__(self, book_id, title, author, borrower, days_late):
        self.book_id = book_id
        self.title = title
        self.author = author
        self.borrower = borrower
        self.days_late = days_late

    def display_details(self):
        print(f"Book ID: {self.book_id}")
        print(f"Title: {self.title}")
        print(f"Author: {self.author}")
        print(f"Borrower: {self.borrower}")
        print(f"Days Late: {self.days_late}")

    def calculate_late_fee(self):
        if self.days_late <= 5:
            late_fee = self.days_late * 5
        elif 6 <= self.days_late <= 10:
            late_fee = self.days_late * 7
        else:
            late_fee = self.days_late * 10
        print(f"Late Fee: {late_fee}")

librarybook1 = LibraryBook(501, "To Kill a Mockingbird", "Harper Lee", "Sarah Smith", 4)
librarybook1.display_details()
librarybook1.calculate_late_fee()

```

Output:

```

PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/OneDrive/Desktop/ai/day7.py
Book ID: 501
Title: To Kill a Mockingbird
Author: Harper Lee
Borrower: Sarah Smith
Days Late: 4
Late Fee: 20
PS C:\Users\rohit\OneDrive\Desktop\ai>

```

Task 5:

Student Performance Report - Define a function

student_report(student_data) that accepts a dictionary containing student names and their marks. The function should:

- Calculate the average score for each student
- Determine pass/fail status (pass ≥ 40)
- Return a summary report as a list of dictionaries

Use Copilot suggestions as you build the function and format the

Code:

```
118 def student_performance_report(students):
119     report = {}
120     for name, marks in students.items():
121         if marks >= 40:
122             report[name] = "Pass"
123         else:
124             report[name] = "Fail"
125
126 def calculate_average_marks(students):
127     averages = {}
128     for name, marks_list in students.items():
129         averages[name] = sum(marks_list) / len(marks_list)
130     return averages
131
132 students = {
133     "Alice": [85, 90, 78],
134     "Bob": [35, 40, 50],
135     "Charlie": [60, 70, 80]
136 }
137
138 average_marks = calculate_average_marks(students)
139 print("Average marks:")
140 for name, avg in average_marks.items():
141     print(f"{name}: {avg:.2f}")
142
143 performance_report = {
144     name: "Pass" if all(mark >= 40 for mark in marks) else "Fail"
145     for name, marks in students.items()
146 }
147
148 print("Student performance report:")
149 for name, status in performance_report.items():
150     print(f"{name}: {status}")
151
```

Output:

```
PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/OneDrive/Desktop/ai/day7.py
Average marks:
Alice: 84.33
Bob: 41.67
Charlie: 70.00
Student performance report:
Alice: Pass
Bob: Fail
Charlie: Pass
PS C:\Users\rohit\OneDrive\Desktop\ai>
```


Task 6:

Taxi Fare Calculation-Create Python code that defines a class named TaxiRide with attributes: ride_id, driver_name, distance_km, and waiting_time_min. Implement a method display_details() to print ride details, and a method calculate_fare() where:

- ₹15 per km for the first 10 km
- ₹12 per km for the next 20 km
- ₹10 per km above 30 km
- Waiting charge: ₹2 per minute

Create a ride object, display details, and print the total fare.

Code:

```
153 class TaxiRide:
154     def __init__(self, ride_id, driver_name, distance_km, waiting_time_min):
155         self.ride_id = ride_id
156         self.driver_name = driver_name
157         self.distance_km = distance_km
158         self.waiting_time_min = waiting_time_min
159
160     def display_details(self):
161         print(f"Ride ID: {self.ride_id}")
162         print(f"Driver Name: {self.driver_name}")
163         print(f"Distance (km): {self.distance_km}")
164         print(f"Waiting Time (min): {self.waiting_time_min}")
165
166     def calculate_fare(self):
167         if self.distance_km <= 10:
168             fare = self.distance_km * 15
169         elif 11 <= self.distance_km <= 30:
170             fare = (10 * 15) + (self.distance_km - 10) * 12
171         else:
172             fare = (10 * 15) + (20 * 12) + (self.distance_km - 30) * 10
173
174         fare += self.waiting_time_min * 2
175         return fare
176
177 ride = TaxiRide(501, "Charlie Brown", 25, 10)
178 ride.display_details()
179 fare = ride.calculate_fare()
180 print(f"Total Fare: {fare}")
181
```

Output:

```
PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/OneDrive/Desktop/ai/day7.py
Ride ID: 501
Driver Name: Charlie Brown
Distance (km): 25
Waiting Time (min): 10
Total Fare: 350
PS C:\Users\rohit\OneDrive\Desktop\ai>
```

Task 7:

Statistics Subject Performance - Create a Python function

`statistics_subject(scores_list)` that accepts a list of 60 student scores and computes key performance statistics. The function should return the following:

- Highest score in the class
- Lowest score in the class
- Class average score
- Number of students passed (score ≥ 40)
- Number of students failed (score < 40)

Allow Copilot to assist with aggregations and logic

Code:

```
183 def statistics_subject(score_list):
184     total = sum(score_list)
185     average = total / len(score_list)
186     highest = max(score_list)
187     lowest = min(score_list)
188     passed = 0
189     failed = 0
190
191     for i in score_list:
192         if i >= 40:
193             passed += 1
194         else:
195             failed += 1
196
197     print(f"Number of Students Passed: {passed}")
198     print(f"Number of Students Failed: {failed}")
199
200     return {
201         "average": average,
202         "highest": highest,
203         "lowest": lowest
204     }
205
206 scores = [
207     28, 49, 33, 72, 15, 60, 95, 40, 51, 81, 22, 47, 68, 79, 34, 91, 44, 58,
208     73, 38, 66, 84, 29, 50, 77, 92, 41, 36, 65, 80, 54, 87, 30, 69, 45, 71,
209     39, 83, 59, 74
210 ]
211
212 stats = statistics_subject(scores)
213 print(stats)
214 |
```

Output:

```
PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/OneDrive/Desktop/ai/day7.py
Number of Students Passed: 30
Number of Students Failed: 10
{'average': 57.725, 'highest': 95, 'lowest': 15}
PS C:\Users\rohit\OneDrive\Desktop\ai>
```

Task Description #8 (Transparency in Algorithm Optimization)

Task: Use AI to generate two solutions for checking prime numbers:

- Naive approach (basic)
- Optimized approach

Prompt:

“Generate Python code for two prime-checking methods and explain how the optimized version improves performance.”

Expected Output:

- Code for both methods.
- Transparent explanation of time complexity.
- Comparison highlighting efficiency improvements.

Code:

```
185 # generate a program that reads a file and process the data
186 # Generate code with proper error handling and clear explanations for each exception.
187
188 def read_file(file_path):
189     try:
190         # Attempt to open the file
191         with open(file_path, "r") as file:
192             data = file.read()
193             print("File content successfully read.")
194             return data
195
196     except FileNotFoundError:
197         # Handle the case where the file does not exist
198         print(f"Error: The file at {file_path} was not found.")
199
200     except PermissionError:
201         # Handle the case where there are permission issues
202         print(f"Error: You do not have permission to read the file at {file_path}.")
203
204     except Exception as e:
205         # Handle any other exceptions that may occur
206         print(f"An unexpected error occurred: {e}")
207
208 # Specify the correct path to your file
209 file_path = "example.txt"
210
211 file_content = read_file(file_path)
212
213 if file_content:
214     print("File Content:")
215     print(file_content)
216
217
```

Output:

```
File content successfully read.
File Content:
Hello Everyone
Welcome to AI Assisted Coding class
Third year second semester
SR University
Lets work with files as part of lab assignment
```


Task Description #9 (Transparency in Recursive Algorithms)

Objective: Use AI to generate a recursive function to calculate Fibonacci numbers.

Instructions:

1. Ask AI to add clear comments explaining recursion.
2. Ask AI to explain base cases and recursive calls.

Expected Output:

- Well-commented recursive code.
- Clear explanation of how recursion works.
- Verification that explanation matches actual execution.

Code:

```
218 # Write a code to generate a recursive function to calculate fibonacci numbers.
219 # Add clear comments explaining recursion.
220 # Also explain base cases and recursive calls.
221 # Verification that explanation matches actual execution.
222
223 def fibonacci(n):
224     """
225     Calculate the nth Fibonacci number using recursion.
226
227     The Fibonacci sequence is defined as:
228     F(0) = 0 (base case)
229     F(1) = 1 (base case)
230     F(n) = F(n-1) + F(n-2) for n > 1 (recursive case)
231
232     Parameters:
233     n (int): The position in the Fibonacci sequence to calculate.
234
235     Returns:
236     int: The nth Fibonacci number.
237     """
238
239     # Base cases
240     if n == 0:
241         return 0
242     elif n == 1:
243         return 1
244     else:
245         # Recursive case: sum of the two preceding numbers
246         return fibonacci(n - 1) + fibonacci(n - 2)
247
248 # Example usage and verification
249 n = 6
250 print(f"The {n}th Fibonacci number is: {fibonacci(n)}")
251
252 # Explanation:
253 # When calling fibonacci(6), the function checks if n is 0 or 1.
254 # Since it is neither, it proceeds to the recursive case:
255 # fibonacci(6) = fibonacci(5) + fibonacci(4)
256 # This pattern continues, breaking down each call until it reaches the base cases.
257 # fibonacci(1) returns 1 and fibonacci(0) returns 0.
258 # The results are then combined step by step to produce the final answer.
259
```

Output:

```
PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/OneDrive/Desktop/ai/day7.py
The 6th Fibonacci number is: 8
PS C:\Users\rohit\OneDrive\Desktop\ai>
```

Task Description #10 (Transparency in Error Handling)

Task: Use AI to generate a Python program that reads a file and processes data.

Prompt:

“Generate code with proper error handling and clear explanations for each exception.”

Expected Output:

- Code with meaningful exception handling.
- Clear comments explaining each error scenario.
- Validation that explanations align with runtime behavior.

Code:

```
260
261 import time
262
263 def is_prime_naive(n):
264     if n <= 1:
265         return False
266     for i in range(2, n):
267         if n % i == 0:
268             return False
269     return True
270
271 start_time = time.time()
272 number = 29
273 result_naive = is_prime_naive(number)
274 end_time = time.time()
275
276 print(f"Naive Approach: Is {number} prime? {result_naive}")
277 print(f"Time taken (Naive): {end_time - start_time} seconds")
278 |
```

Output:

```
PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/OneDrive/Desktop/ai/day7.py
Naive Approach: Is 29 prime? True
Time taken (Naive): 0.0 seconds
PS C:\Users\rohit\OneDrive\Desktop\ai>
```

Ln 278, Col 1